PACKAGE DELIVERY TRACKER

Projektoverblik, arkitektur og nuværende status

Formål

Et Java-baseret backendprojekt, hvor brugere kan oprette pakker, følge deres status og se leveringshistorik. Projektet kombinerer Javalin, Hibernate/JPA, PostgreSQL, en ekstern adresse-API og WebSockets.

Dokumentet beskriver det, der faktisk findes i projektet nu. Færdige dele og påbegyndte dele er markeret tydeligt.

1. Projektets opbygning

Den planlagte arkitektur følger en lagdelt struktur. Hvert lag har ét hovedansvar:

HTTP / WebSocket	Routes	Controller	Service	DAO	PostgreSQL
Request	URL'er	HTTP-håndtering	Regler	Databasekald	Data

Routes og serveropsætning findes. Controller, service og DAO er endnu ikke koblet sammen, så den fulde kæde er ikke færdig.

Teknologier

- Java 25 og Maven som projektgrundlag.
- Javalin 7 til REST-endpoints og WebSockets.
- Hibernate/JPA til ORM og PostgreSQL som database.
- Jackson til konvertering mellem JSON og Java-objekter.
- Lombok til getters, setters og tomme constructors.
- JUnit og Testcontainers er tilføjet som testafhængigheder.
- SLF4J og Logback er tilføjet til logging.

2. Domænemodellen

Projektet har tre centrale entities. De bliver registreret hos Hibernate og svarer til databasetabeller.

Entity	Ansvar	Vigtig relation
User	Konto, email, password-hash og rolle.	Én bruger kan eje flere pakker.
Parcel	Trackingnummer, adressering, koordinater og aktuel status.	Tilhører én bruger.
DeliveryEvent	En tidsstemplet ændring i pakkens leveringshistorik.	Tilhører én pakke.

3. Entities og forretningsdata

User

User gemmes i tabellen users. ID'et genereres af databasen, email skal være unik, og rollen gemmes som tekst. Password-feltet hedder passwordHash, fordi et rigtigt password senere skal hashes før lagring.

```
Roller: USER, EMPLOYEE og ADMIN.
```

Parcel

Parcel gemmes i tabellen packages. Trackingnummeret skal være unikt. Pakken indeholder afsender, modtager, destinationsadresse, koordinater, aktuel status, oprettelsestidspunkt og ejer.

```
En ny Parcel får automatisk status CREATED og tidspunktet Instant.now() gennem constructoren.
```

DeliveryEvent

DeliveryEvent gemmer historikken. Det har status, lokation, besked og timestamp. Relationen til Parcel er LAZY, så den tilknyttede pakke ikke altid indlæses med det samme.

```
@PrePersist sørger for et timestamp, hvis feltet stadig er null lige før indsættelsen.
```

Statusforløb

```
CREATED -> PICKED_UP -> IN_TRANSIT -> AT_SORTING_CENTER -> OUT_FOR_DELIVERY -> DELIVERED
```

CANCELLED findes også. Regler for lovlige og ulovlige statusskift er endnu ikke implementeret.

4. DTO-laget

DTO'er bruges som API'ets input og output. De holder JSON-formatet adskilt fra database-entities.

CreateParcelDTO

Skal modtage data ved oprettelse af en pakke.

ParcelDTO

Skal sende en pakke tilbage til klienten uden at eksponere entity'en direkte.

DeliveryEventDTO

Kopierer et DeliveryEvent til et egnet API-svar.

OpenRouteDTO

Skal repræsentere relevante dele af OpenRouteService-responsen, men er lige nu tom.

ValidatedAddress

Skal samle formatteret adresse, latitude og longitude i et enkelt resultat.

Vigtig navneforskel

CreateParcelDTO bruger ownerID, receiverName og destinationAdress. HTTP-eksemplet bruger ownerId, recipientName og destinationAddress. Navnene bør være ens, så Jackson kan mappe JSON korrekt.

5. Databasekonfiguration

HibernateConfig opretter én delt EntityManagerFactory. Double-checked locking med volatile og synchronized forhindrer, at flere tråde opretter hver sin factory.

- Lokalt læses databasenavn, brugernavn og password fra config.properties.
- Ved deployment læses værdierne fra environment variables.
- HibernateBaseProperties indstiller PostgreSQL-driveren og HikariCP.
- EntityRegistry registrerer User, Parcel og DeliveryEvent.
- HikariCP kan have op til 10 forbindelser og mindst 2 ledige forbindelser.

> **Udviklingsindstilling med risiko**
>
> hibernate.hbm2ddl.auto står til create. Når Hibernate-factory'en starter, kan tabellerne blive genskabt og eksisterende data forsvinde. Det er nyttigt tidligt i udviklingen, men ikke til produktion.

6. REST-server og routes

Main opretter ParcelRoutes, Routes og ApplicationConfig og starter Javalin på port 7070. En shutdown hook stopper serveren, når JVM'en afsluttes.

Adresse	Nuværende funktion
`GET /api/`	Returnerer 'Package Tracker API'.
`GET /api/parcels/`	Returnerer 'Parcel Routes virker'.
`WS /api/parcels/{trackingNumber}/live`	Opretter en simpel WebSocket-forbindelse.

Der findes endnu ikke POST, PUT eller DELETE endpoints til pakker. HTTP-filen kalder /api/packages, mens den registrerede route hedder /api/parcels.

7. WebSocket

WebSocket-endpointet har onConnect, onMessage, onClose og onError. onMessage sender lige nu klientens besked tilbage til samme klient. Det er en echo-test, ikke en rigtig trackingudsendelse.

For rigtig live tracking skal TrackingSocket senere gemme forbindelser pr. trackingnummer. Når en service opdaterer en pakkes status, sendes den nye status til de klienter, der følger pakken.

8. Lag, der stadig er under udvikling

DAO

UserDAO og ParcelDAO er tomme. De skal indeholde databaseoperationer og transaktioner.

Service

ParcelService er et skelet. Trackingnummer, adressevalidering, statusregler og første event mangler.

Controller

ParcelController afhænger lige nu af en anden ParcelController. Den bør senere forbindes til ParcelService og have HTTP-handlers.

OpenRoute

OpenRouteService har URL, API-nøgle, HttpClient og ObjectMapper, men validate() returnerer null.

Security

Der findes roller, men endnu ingen registrering, login, hashing, JWT eller routebeskyttelse.

Deployment

Environment-konfiguration er forberedt, men selve deployment-opsætningen mangler.

9. Fejlhåndtering og logging

ApiException kan indeholde en statuskode og en besked og logger fejlen gennem SLF4J. ApplicationConfig har dog endnu ingen ApiException-handler, så koden bliver ikke automatisk lavet om til det ønskede HTTP-svar.

ValidationException bliver returneret som 400 Bad Request. Beskeden siger altid, at et path-parameter skal være et tal, selv om valideringsfejlen kan have en anden årsag.

10. Teststatus

DatabaseTest forsøger at gemme User, Parcel og DeliveryEvent i én transaktion og ruller tilbage ved fejl. Det er et godt eksempel på transaktionshåndtering, men filen er ikke en rigtig JUnit-test endnu.

Aktuel konflikt

DatabaseTest bruger User.builder(), Parcel.builder() og DeliveryEvent.builder(). @Builder er fjernet fra entities, så testen matcher ikke længere koden.

En compile-kontrol kunne ikke køres fra terminalen, fordi mvn ikke er installeret som kommando, og projektet ikke indeholder Maven Wrapper.

11. Samlet status

Område	Status
Maven og dependencies	Oprettet
Javalin-server	Oprettet
Entities og enums	Overvejende oprettet
Hibernate-konfiguration	Oprettet
Route-struktur	Oprettet
WebSocket echo	Oprettet
DTO'er	Delvist færdige
DAO	Mangler
ParcelService	Mangler
ParcelController	Mangler
CRUD-endpoints	Mangler
OpenRoute-integration	Mangler
JUnit- og REST-tests	Mangler
JWT og adgangskontrol	Mangler
Deployment	Mangler

12. Anbefalet rækkefølge

1. Ensret feltnavne i DTO'er, entities og HTTP-eksempler.
2. Ret ParcelDTO, så constructoren læser fra Parcel-parameteren.
3. Implementer ParcelDAO med create, find og update.
4. Implementer ParcelService og de centrale forretningsregler.
5. Lad ParcelController afhænge af ParcelService.
6. Forbind CRUD-handlers til ParcelRoutes.
7. Færdiggør OpenRouteDTO og OpenRouteService.validate().
8. Lav rigtige DAO- og REST-tests.
9. Tilføj password-hashing, JWT og rollebaseret adgang.
10. Forbind WebSocket-udsendelser med statusændringer.

Sikkerhed

OpenRouteService-nøglen ligger direkte i config.properties og har været delt. Den bør tilbagekaldes og erstattes. Den nye nøgle bør gemmes som environment variable og aldrig commits til Git.

Konklusion

Projektet har et godt teknisk fundament: domænet er defineret, Hibernate er konfigureret, og Javalin-serveren kan starte med routes og en WebSocket-test. Næste milepæl er at forbinde DAO, service, controller og routes, så en pakke faktisk kan oprettes, gemmes og hentes gennem REST-API'et.